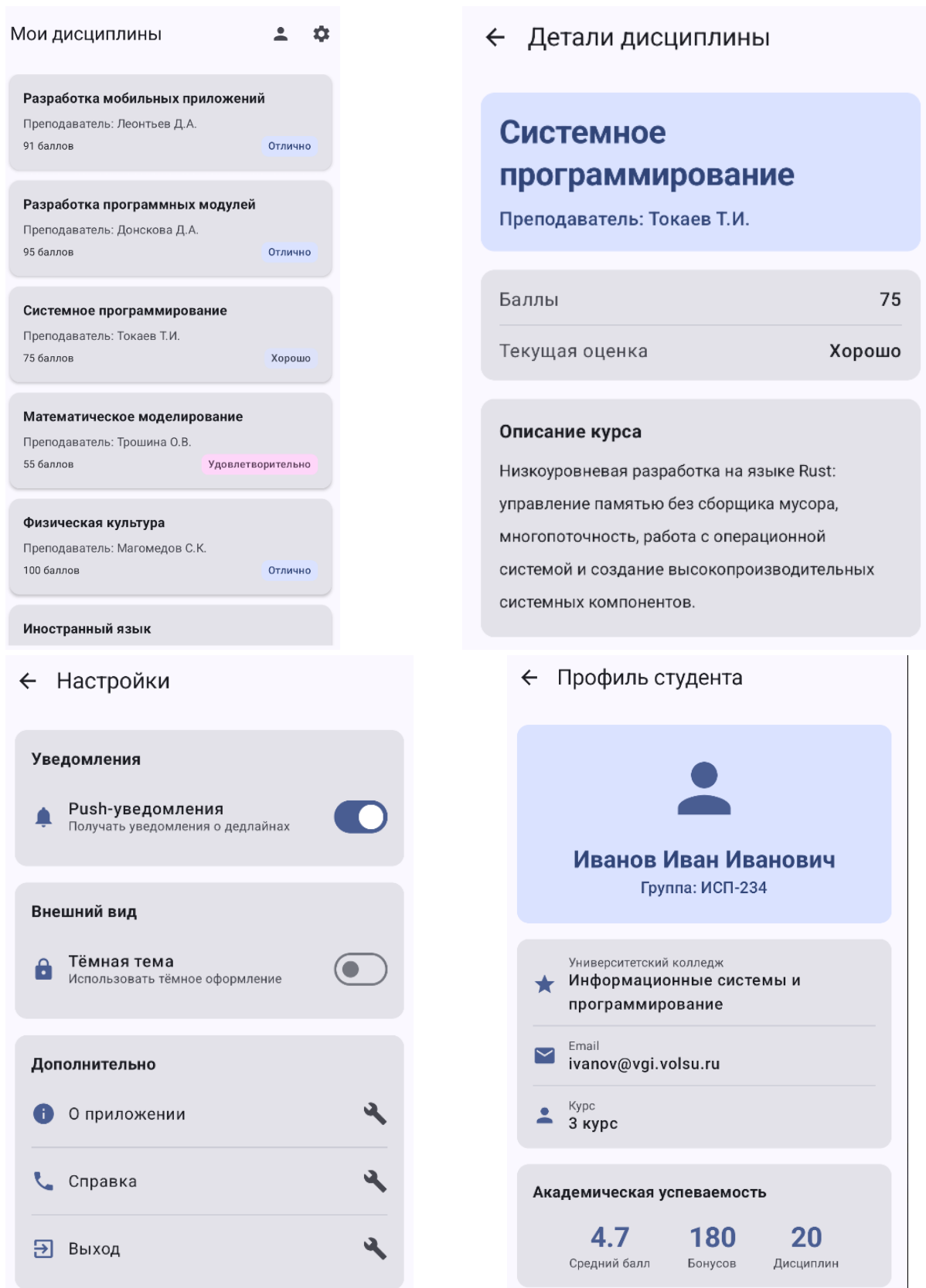


Лабораторная работа №15-16. Navigation in Jetpack Compose

Цель работы: Изучить принципы навигации в приложениях на Jetpack Compose с использованием библиотеки Navigation Compose. Научиться создавать многоэкранные приложения, передавать данные между экранами и управлять стеком навигации. Разработать приложение "Студенческий планер" с несколькими экранами для демонстрации различных типов навигации.

Пример реализации:



Теоретическая часть. Что такое Navigation?

Навигация — это процесс перехода пользователя между различными экранами (destinations) вашего приложения. В традиционной Android-разработке для этого использовались **Activity** и **Fragments**, но в **Jetpack Compose** появился более современный подход — **Navigation Compose**.

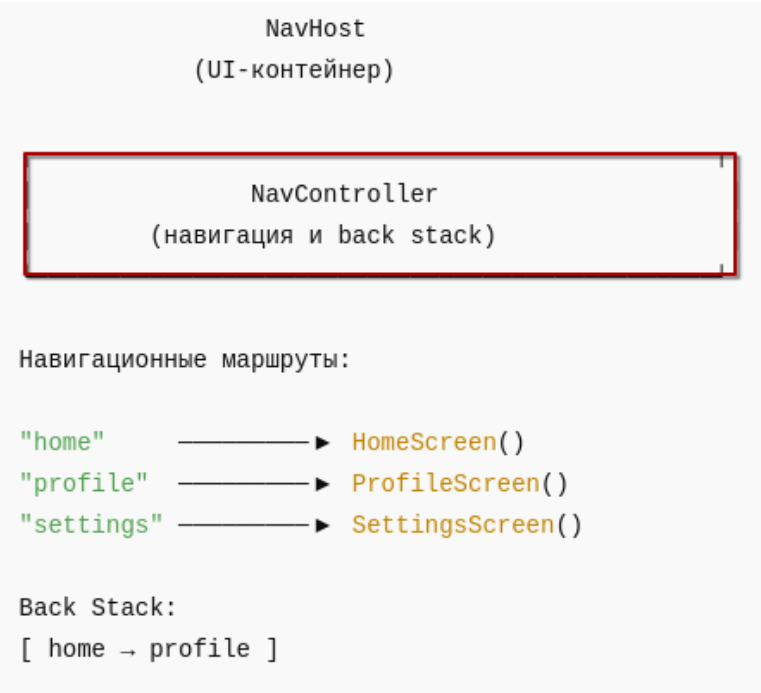
Зачем нужна навигация?

- Большинство приложений содержат больше одного экрана
- Пользователь должен легко перемещаться между экранами
- Кнопка "Назад" должна работать правильно
- Нужно сохранять состояние экранов при переходах
- Нужно передавать данные между экранами

Ключевые концепции Navigation Compose

Концепция	Описание	Пример
NavHost	Контейнер для навигации, отображает текущий экран	Один на всё приложение
NavController	Управляет навигацией (переходы, back stack)	navController.navigate("details")
Route	Уникальный идентификатор экрана (строка)	"home", "profile/{userId}"
Destination	Конечная точка навигации (экран)	Composable-функция
Arguments	Данные, передаваемые между экранами	ID пользователя, название товара
Back Stack	Стек посещённых экранов	Home → Details → Settings

Как работает Navigation?



Процесс навигации:

1. **Пользователь нажимает кнопку** → "Открыть профиль"
2. **Код вызывает** → `navController.navigate("profile")`
3. **NavController добавляет** → "profile" в back stack
4. **NavHost отображает** → `ProfileScreen()`
5. **Кнопка "Назад"** → возвращает к предыдущему экрану из back stack

Типы навигации

1. Простая навигация (без параметров):

// Определение маршрута

```
composable("home") { HomeScreen() }
```

// Переход

```
navController.navigate("home")
```

2. Навигация с обязательными параметрами:

// Определение маршрута

```
composable("profile/{userId}") { backStackEntry ->
    val userId = backStackEntry.arguments?.getString("userId")
    ProfileScreen(userId)
}
```

// Переход

```
navController.navigate("profile/12345")
```

3. Навигация с опциональными параметрами:

// Определение маршрута

```
composable(
    "search?query={query}",
    arguments = listOf(navArgument("query") {
        defaultValue = ""
    })
) { backStackEntry ->
    val query = backStackEntry.arguments?.getString("query")
    SearchScreen(query)
}
```

// Переход

```
navController.navigate("search?query=kotlin")
```

Управление Back Stack

Back Stack — это стек (LIFO - Last In First Out) экранов, по которым переходил пользователь.

Пример back stack:

Settings	← Текущий экран
Profile	
Details	
Home	← Начальный экран

Кнопка "**Назад**" → убирает **Settings**, показывает **Profile**

Практическая часть. Подготовка к работе

Создание проекта

1. Откройте **Android Studio**
2. Выберите **File** → **New** → **New Project**
3. Выберите **Empty Activity** → **Next**

Empty Activity

Create a new empty activity with Jetpack Compose

Name: Student Planner

Package name: com.leontev.studentplanner

Save location: /home/denis/Project/AndroidStudioProjects/StudentPlanner

Minimum SDK: API 25 ("Nougat"; Android 7.1.1)

Info Your app will run on approximately 98,8% of devices.
[Help me choose](#)

Build configuration language *?* Kotlin DSL (build.gradle.kts) [Recommended]

Previous Next Cancel **Finish**

4. Укажите параметры:

- Name: **Student_Planner**
- Package name: **com.baure_фюо.studentplanner**
- Minimum SDK: API 25

5. Нажмите **Finish**

Дождитесь загрузки проекта.

Создание Git-репозитория

Перед началом работы создайте Git-репозиторий для отслеживания изменений и защиты от потери данных.

Вариант 1: Через Android Studio (рекомендуется)

1. В Android Studio выберите **VCS → Enable Version Control Integration**
2. Выберите **Git → OK**
3. Выберите **Git → Commit** (или **Ctrl+K**)
4. **Выберите все файлы**, введите сообщение: **"Initial commit"**
5. Нажмите **Commit**

Вариант 2: Через терминал

1. Откройте терминал в Android Studio (**View → Tool Windows → Terminal**)
2. Выполните команды:

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория на GitHub

1. Откройте **GitHub** и войдите в аккаунт.
2. Нажмите **New repository** (зелёная кнопка)
3. **Repository name:** **Student_Planner_FIO** (где FIO — ваша фамилия)
4. Выберите **Public** (публичный репозиторий)
5. **НЕ** ставьте галочки на "Add README" и ".gitignore"
6. Нажмите **Create repository**
7. Скопируйте URL репозитория

Привязка локального репозитория к удалённому

В терминале Android Studio выполните:

git branch -M main

git remote add origin <URL_репозитория>

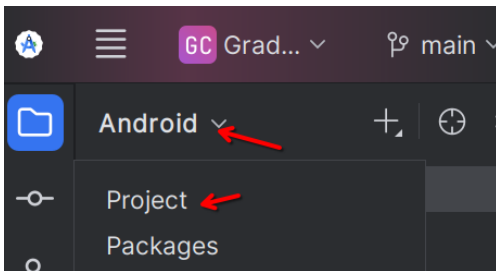
git push -u origin main

* Замените **<URL_репозитория>** на скопированный URL

При первом **push** вас попросят ввести логин и пароль **GitHub**.

3. Создание папки **img**

1. В Android Studio переключитесь в режим просмотра **Project** (в левом верхнем углу панели Project)



2. Найдите **app**

3. Кликните правой кнопкой на папке **app**

4. Выберите **New** → **Directory**

5. Введите имя: **img**

6. Нажмите **OK**

Полный путь к папке: `/app/img/`

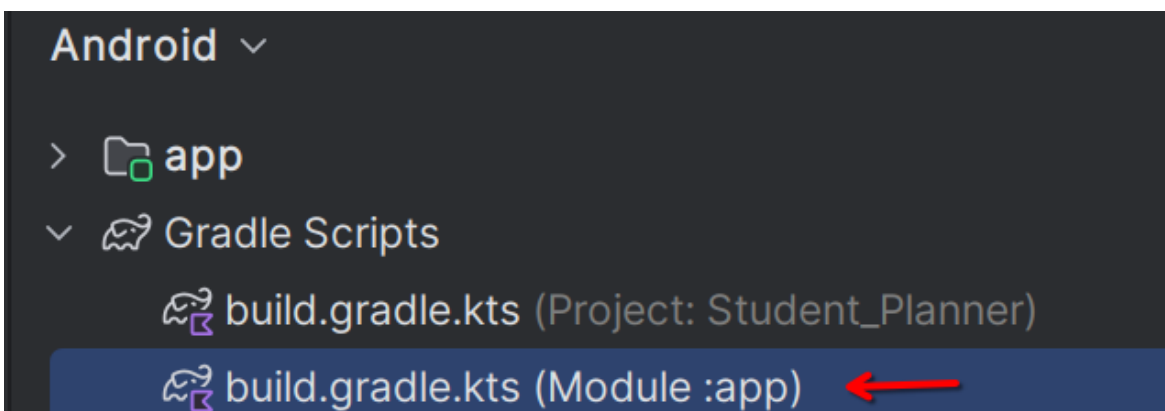
Шаг 1. Добавление зависимостей **Navigation Compose**

Чтобы использовать навигацию в Jetpack Compose, нужно подключить специальную библиотеку.

1.1. Открытие файла **build.gradle**

1. В левой панели Android Studio (режим **Android**)

2. Откройте **Gradle Scripts** → **build.gradle.kts (Module :app)**



3. Найдите блок **dependencies { ... }**

1.2. Добавление зависимости **Navigation**

Добавляем библиотеку **Navigation Compose**, которая предоставляет **NavController**, **NavHost** и другие компоненты для навигации.

Добавьте следующую строку в блок **dependencies**:

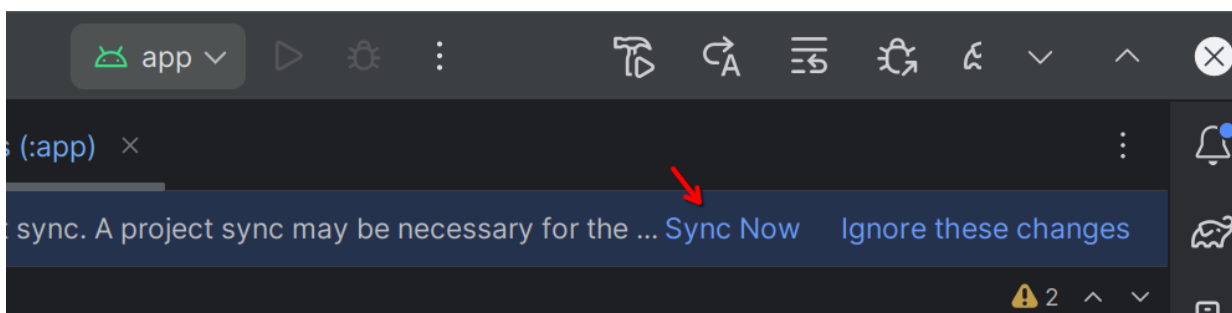
```
implementation("androidx.navigation:navigation-compose:2.7.6")
```

Объяснение:

- navigation-compose:2.7.6 — библиотека навигации для Jetpack Compose
- актуальную версию можно узнать на официальном сайте developer.android.com (на момент написания лабораторной работы - 2.9.7, но для лучшей совместимости мы будем использовать более стабильную)

1.3. Синхронизация проекта

1. В правом верхнем углу появится панель **"Sync Now"**
2. Нажмите **"Sync Now"** и дождитесь завершения



3. Убедитесь, что синхронизация прошла успешно (в нижней панели не должно быть ошибок)

Важно: Если возникли ошибки, проверьте версию библиотеки и подключение к интернету.

Скриншоты для отчёта



Сделайте скриншот:

- Открытый файл build.gradle.kts с добавленной зависимостью
- Успешная синхронизация (сообщение в нижней панели)
- Сохраните как: **step1_dependencies_FIO.png**

Выполните в терминале:

```
git add .
```

```
git commit -m "Step 1: Add Navigation Compose dependency"
```

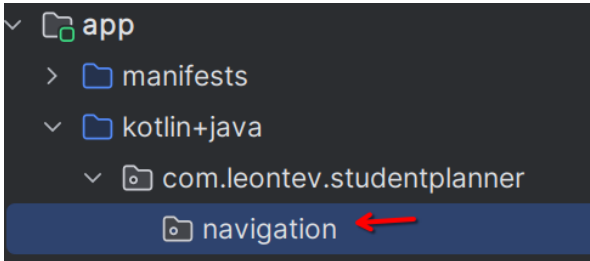
```
git push
```

Шаг 2. Создание sealed class для маршрутов

Создадим централизованное хранилище всех маршрутов приложения для **type-safety**.

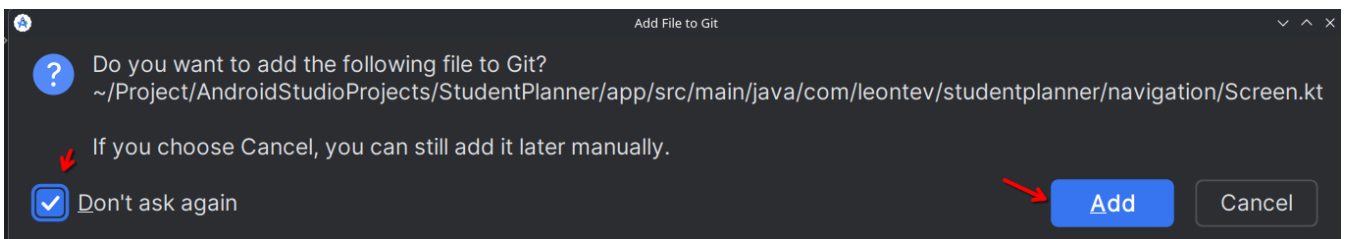
2.1. Создание пакета navigation

1. В режиме **Android** найдите пакет **com.ваше_фио.studentplanner**
2. Правой кнопкой мыши → **New** → **Package**
3. Введите: **navigation**
4. Нажмите **OK**

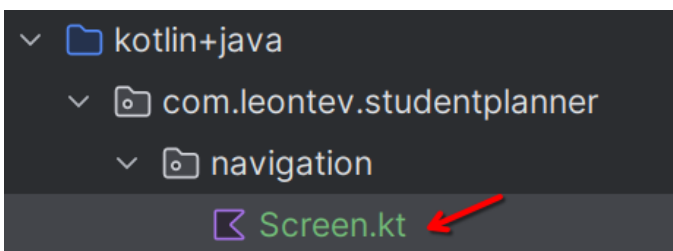


2.2. Создание файла Screen

1. Правой кнопкой на пакет **navigation** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **Screen**
4. Нажмите **Enter**
5. Соглашаемся на отслеживание файла в Git:



Итог:



2.3. Определение маршрутов

Открываем файл **Screen.kt**. В нём создаём **sealed class**, который описывает все возможные экраны нашего приложения. **Sealed class** гарантирует, что компилятор знает о всех возможных вариантах экранов.

1. Добавьте следующий код:

```
sealed class Screen(val route: String) {  
}
```


- **sealed class Screen(val route: String)** — базовый класс для всех экранов. Каждый экран будет иметь свойство `route` (строковый идентификатор)

2. Внутри класса **Screen** создадим — главный экран - список дисциплин:

```
object Home : Screen("home")
```

- **object Home : Screen("home")** — экран "Главная". Используем `object` потому что нужен только один экземпляр

3. Внутри класса **Screen** создадим — экран детальной информации о дисциплине, который будет принимать параметр **subjectId**:

```
object Details : Screen("details/{subjectId}") {  
    fun createRoute(subjectId: String) = "details/$subjectId"  
}
```

- **object Details : Screen("details/{subjectId}")** — экран "Детали". В маршруте `{subjectId}` — это **placeholder** для параметра (ID дисциплины)
- **fun createRoute(subjectId: String)** — вспомогательная функция для создания маршрута с конкретным ID. Например: `Details.createRoute("123")` вернёт `"details/123"`

4. Внутри класса **Screen** создадим — экран профиля студента:

```
object Profile : Screen("profile")
```

5. Внутри класса **Screen** создадим — экран настроек:

```
object Settings : Screen("settings")
```

Скриншоты для отчёта



Сделайте скриншот:

- Созданного файла **Screen.kt** с кодом
- Структура пакетов слева (должен быть виден пакет **navigation**)
- Сохраните как: **step2_screen_routes_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 2: Create Screen sealed class for routes"

git push

Шаг 3. Создание моделей данных

Создадим модель данных для дисциплин, с которыми будем работать.

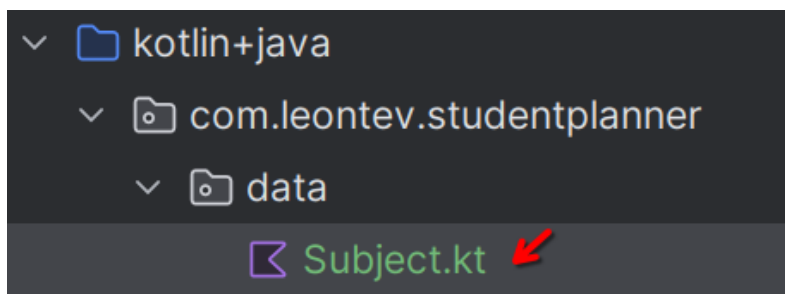
3.1. Создание пакета data

1. Правой кнопкой на пакет **com.ваше_фio.studentplanner** → **New** → **Package**
2. Введите: **data**
3. Нажмите **OK**

3.2. Создание файла Subject

1. Правой кнопкой на пакет **data** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **Subject**
4. Нажмите **Enter**

Итог:



3.3. Определение модели данных

Открываем файл **Subject.kt**. В нём создаём **data class** для представления учебной дисциплины с информацией о названии, преподавателе, количестве баллов и текущей оценке.

1. Создаём **data class Subject** — модель учебной дисциплины:

```
data class Subject(  
    val id: String, // Уникальный идентификатор  
    val name: String, // Название дисциплины  
    val professor: String, // Имя преподавателя  
    val credits: Int, // Количество баллов  
    val currentGrade: String, // Текущая оценка  
    val description: String // Описание курса  
)
```

В текущем файле создадим список тестовых данных. **Текст для вставки:**

```
id = "1",
name = "Разработка мобильных приложений",
professor = "Леонтьев Д.А.",
credits = 91,
currentGrade = "Отлично",
description = "Разработка нативных Android-приложений с использованием Kotlin и современных компонентов Jetpack Compose. Изучение архитектурных паттернов MVVM, работы с локальными базами данных и сетевыми API."
```

Как должен выглядеть список:

```
val sampleSubjects = listOf(
    Subject(
        id = "1",
        name = "Разработка мобильных приложений",
        professor = "Леонтьев Д.А.",
        credits = 91,
        currentGrade = "Отлично",
        description = "Разработка нативных Android-приложений с использованием Kotlin и современных компонентов Jetpack Compose. Изучение архитектурных паттернов MVVM, работы с локальными базами данных и сетевыми API."
    ),
)
```

Ещё один пример. Текст для вставки:

```
id = "3",
name = "Системное программирование",
professor = "Токаев Т.И.",
credits = 75,
currentGrade = "Хорошо",
description = "Низкоуровневая разработка на языке Rust: управление памятью без сборщика мусора, многопоточность, работа с операционной системой и создание высокопроизводительных системных компонентов."
```

В итоге вы передаёте несколько **data-классов** в список (не забывайте ставить запятую):

```
val sampleSubjects = listOf(
    Subject(...), 1
    Subject(...), 2
    Subject(...), 3
    Subject(...), 4
    Subject(...), 5
    Subject(...), 6
)
```

Ваша задача в конце лабораторной работы создать список всех дисциплин, которые вы изучаете в текущем семестре, с подробным описанием.

Зачем нужны тестовые данные?

- Можно сразу протестировать UI без подключения к реальной базе данных
- Удобно для демонстрации и отладки
- Позже легко заменить на реальные данные из БД

Скриншоты для отчёта



Сделайте скриншот:

- Файл **Subject.kt** с моделью данных и тестовыми дисциплинами
- Сохраните как: **step3_data_model_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 3: Create Subject data model and sample data"

git push

Шаг 4. Создание экрана Home (главный экран)

Создадим главный экран приложения, который будет отображать список дисциплин.

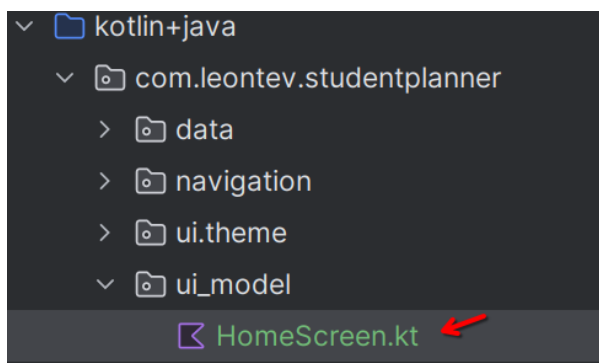
4.1. Создание пакета ui

1. Правой кнопкой на пакет **com.ваше_фио.studentplanner** → **New** → **Package**
2. Введите: **ui_model**
3. Нажмите **OK**

4.2. Создание файла HomeScreen

1. Правой кнопкой на пакет **ui_model** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **HomeScreen**
4. Нажмите **Enter**

Итог:



4.3. Создание базовой структуры экрана

Открываем файл **HomeScreen.kt**. В нём создаём **Composable**-функцию для главного экрана, который отображает список дисциплин в виде карточек.

Добавим в начало файла **необходимые импорты** (обратите внимание на **ваши пакеты**):

```
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.ваш_пакет.studentplanner.data.Subject
import com.ваш_пакет.studentplanner.data.sampleSubjects
```

Создадим карточку отдельной дисциплины

1. Создаём **@Composable**-функцию **SubjectCard()**:

```
@Composable
fun SubjectCard(
    subject: Subject,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) { }
```

- **subject: Subject** - Источник данных — компонент ничего не хранит внутри, он отображает то, что ему передали.
- **onClick: () -> Unit** - **onClick: () -> Unit** - **Callback** для событий — карточка не решает, что делать при клике (открыть экран? показать диалог?). Она лишь уведомляет родителя через переданную функцию. Это делает компонент независимым от навигации.
- **modifier: Modifier = Modifier** - Расширяемость через композицию — позволяет родителю задавать отступы, анимации, ограничения размера без изменения кода компонента. Значение по умолчанию **Modifier** упрощает использование.

2. Корневой контейнер — кликабельная карточка

Внутри тела функции **SubjectCard()** добавьте:

```

@Composable
fun SubjectCard(...) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) { }
}

```

- **.fillMaxWidth()** — карточка растягивается на всю доступную ширину списка. Без этого карточки будут «прилипать» к левому краю.
- **.clickable { }** — важно: кликабельность добавлена **на уровне карточки**, а не на внутренних элементах. Это:
 - Улучшает UX (большая зона клика)
 - Генерирует визуальную обратную связь (рипл-эффект) по всей площади
 - Избегает «мертвых зон» между текстовыми полями
- **elevation** — тень создаёт ощущение «поднятости» элемента над фоном (принцип **Material Design**). Значение **2.dp** — стандарт для списков.

3. Внутрь карточки Card добавьте следующую структуру:

```

) {
    Card(...) {
        1 Column(...) {
            // Название дисциплины
            2 Text(...)
            3 Spacer(modifier = Modifier.height(8.dp))
            // Преподаватель
            4 Row(...) {
                5 Text(...)
            }
            6 Spacer(modifier = Modifier.height(4.dp))
            // Баллы и оценка
            7 Row(...) {
                8 Text(...)
                // Цветная метка с оценкой
                9 Surface(...) {
                    10 Text(...)
                }
            }
        }
    }
}

```

3.1. Внутренняя структура — **Column** с отступами

```
Column(  
    modifier = Modifier.padding(16.dp)  
) {
```

- Элементы располагаются **вертикально** (название → преподаватель → кредиты/оценка)
- Естественное чтение сверху вниз для карточек с информацией

Важно: padding(16.dp) применяется *внутри* Card, поэтому содержимое не «прилипает» к границам карточки. Это базовый принцип визуальной иерархии.

3.2. Название дисциплины

```
// Название дисциплины  
Text(  
    text = subject.name,  
    style = MaterialTheme.typography.titleMedium,  
    fontWeight = FontWeight.Bold  
)
```

- **titleMedium** — заголовок средней важности. При смене темы (светлая/тёмная) размер/цвет подстроится автоматически.

3.3. Фиксированный вертикальный отступ

```
Spacer(modifier = Modifier.height(8.dp))
```

- **Spacer** — предпочтительнее padding между элементами: визуально разделяет блоки, не привязан к конкретному элементу.

3.4. Информация о преподавателе

```
Row(  
    modifier = Modifier.fillMaxWidth(),  
    horizontalArrangement = Arrangement.SpaceBetween  
) {
```

- **SpaceBetween** — «растягивает» содержимое: текст слева, потенциальный элемент (например, иконка) — справа. Сейчас справа пусто, но структура готова к расширению (например, добавлению иконки «избранное»).

3.5. Передаём текст со стилями и цветом:

```
Text(
  text = "Преподаватель: ${subject.professor}",
  style = MaterialTheme.typography.bodyMedium,
  color = MaterialTheme.colorScheme.onSurfaceVariant
)
```

3.6. Фиксированный вертикальный отступ

```
Spacer(modifier = Modifier.height(4.dp))
```

3.7. Заполняем всю ширину и применяем выравнивание:

```
Row(
  modifier = Modifier.fillMaxWidth(),
  horizontalArrangement = Arrangement.SpaceBetween,
  verticalAlignment = Alignment.CenterVertically
) {
```

3.8. Выводим количество баллов и применяем стиль:

```
Text(
  text = "${subject.credits} баллов",
  style = MaterialTheme.typography.bodySmall
)
```

3.9. Цветная метка оценки

```
// Цветная метка с оценкой
Surface(
  color = when (subject.currentGrade) {
    "Отлично" -> MaterialTheme.colorScheme.primaryContainer
    "Хорошо" -> MaterialTheme.colorScheme.secondaryContainer
    else -> MaterialTheme.colorScheme.tertiaryContainer
  },
  shape = MaterialTheme.shapes.small
) {
```

- **Surface вместо Box** — правильно семантически: это «поверхность» с фоном, а не просто контейнер
- **Цвета из темы.** При смене темы цвета автоматически адаптируются (например, в тёмной теме станут темнее)
- **when без else для всех случаев**

3.10. Передаём текущую оценку и применяем стили с отступами:

```
Text(
    text = subject.currentGrade,
    modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
    style = MaterialTheme.typography.labelMedium
)
```

Итоговый вид:

```
@Composable
fun SubjectCard(...) {
    Card(...) {
        Column(...) {
            // Название дисциплины
            Text(...)
            Spacer(modifier = Modifier.height(8.dp))
            // Преподаватель
            Row(...) {
                Text(...)
            }
            Spacer(modifier = Modifier.height(4.dp))
            // Кредиты и оценка
            Row(...) {
                Text(...)
                // Цветная метка с оценкой
                Surface(...) {
                    Text(...)
                }
            }
        }
    }
}
```

3. Создадим главный экран приложения - список дисциплин:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
    onSubjectClick: (String) -> Unit,
    onProfileClick: () -> Unit,
    onSettingsClick: () -> Unit,
    modifier: Modifier = Modifier
) { }
```

- **onSubjectClick: (String) -> Unit** - Callback при клике на дисциплину
- **onProfileClick: () -> Unit** - Callback для перехода в профиль
- **onSettingsClick: () -> Unit** - Callback для перехода в настройки

Внутри функции **HomeScreen()** напишем следующий код:

```
fun HomeScreen(...) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Мои дисциплины") },
                actions = {
                    // Кнопка перехода в профиль
                    IconButton(onClick = onProfileClick) {
                        Icon(
                            imageVector = Icons.Default.Person,
                            contentDescription = "Профиль"
                        )
                    }
                    // Кнопка перехода в настройки
                    IconButton(onClick = onSettingsClick) {
                        Icon(
                            imageVector = Icons.Default.Settings,
                            contentDescription = "Настройки"
                        )
                    }
                }
            )
        }
    ) { }
}
```

- **Scaffold** - Базовый шаблон Material 3: управляет взаимным расположением topBar, bottomBar, floatingActionButton и основного контента. Гарантирует, что контент не «уезжает» под панели.
- **paddingValues** - Системная переменная от Scaffold — содержит отступы, равные высоте TopAppBar и другим системным элементам. Без этого контент будет скрыт под шапкой!
- **LazyColumn** - Аналог RecyclerView в XML. Создаёт только видимые на экране элементы — критично для производительности при длинных списках. Для 5–10 элементов разница незаметна, но архитектурно правильно использовать сразу.
- **Arrangement.spacedBy(12.dp)** - Умный отступ между элементами списка — не нужно вручную добавлять Spacer после каждой карточки.

Внутри тела **Scaffold** добавим обработку событий — callback-цепочка:

```
fun HomeScreen(...) {
    Scaffold(...)
    { paddingValues ->
        LazyColumn(
            modifier = modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(sampleSubjects) { subject ->
                SubjectCard(
                    subject = subject,
                    onClick = { onSubjectClick(subject.id) }
                )
            }
        }
    }
}
```

Итоговый вид:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(...) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Мои дисциплины") },
                actions = {
                    // Кнопка перехода в профиль
                    IconButton(onClick = onProfileClick) {
                        Icon(...)
                    }
                    // Кнопка перехода в настройки
                    IconButton(onClick = onSettingsClick) {
                        Icon(...)
                    }
                }
            )
        }
    ) { paddingValues ->
        LazyColumn(...) {
            items(sampleSubjects) { subject ->
                SubjectCard(...)
            }
        }
    }
}
```

Как работает навигация:

1. Пользователь кликает по карточке → срабатывает `onClick` внутри `SubjectCard`
2. `SubjectCard` вызывает переданный callback `onSubjectClick(subject.id)`
3. `HomeScreen` получает `subject.id` и передаёт его в навигационную систему (например, `navController.navigate("details/$id")`)
4. **Важно:** `HomeScreen` тоже не знает, куда навигироваться — он лишь вызывает свой параметр `onSubjectClick`. Это позволяет:
 - Тестировать экран без навигации (передать пустую лямбду)
 - Использовать экран в другом контексте (например, в диалоговом окне)

Скриншоты для отчёта



Сделайте скриншот:

- Файл **HomeScreen.kt** с кодом
- Сохраните как: **step4_home_screen_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 4: Create HomeScreen with subject list"

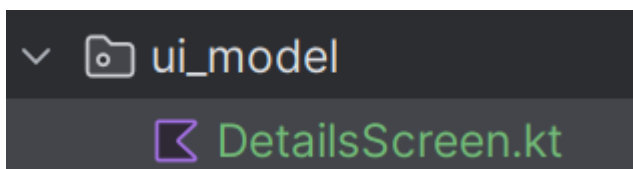
git push

Шаг 5. Создание экрана Details (детали дисциплины)

Создадим экран с подробной информацией о выбранной дисциплине.

5.1. Создание файла DetailsScreen

1. Правой кнопкой на пакет `ui_model` → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **DetailsScreen**
4. Нажмите **Enter**



5.2. Реализация экрана деталей

Открываем файл **DetailsScreen.kt**. В нём создаём экран, который получает ID дисциплины, находит её в списке и отображает подробную информацию.

Добавьте следующий код (**обратите внимание на ваши пакеты**):

```

package com.ваш_пакет.studentplanner.ui_model

import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.ваш_пакет.studentplanner.data.sampleSubjects

// Экран детальной информации о дисциплине
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailsScreen(
    subjectId: String, // ID дисциплины для отображения
    onNavigateBack: () -> Unit, // Callback для возврата назад
    modifier: Modifier = Modifier
) {
    // Находим дисциплину по ID
    val subject = sampleSubjects.find { it.id == subjectId }
    // Если дисциплина не найдена, показываем ошибку
    if (subject == null) {
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = androidx.compose.ui.Alignment.Center
        ) {
            Text("Дисциплина не найдена")
        }
        return
    }
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Детали дисциплины") },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(
                            imageVector = Icons.Default.ArrowBack,
                            contentDescription = "Назад"
                        )
                    }
                }
            )
        }
    ) {
        Column(
            modifier = modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // Карточка с основной информацией
            Card(

```

```

        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ){
        Column(
            modifier = Modifier.padding(16.dp)
        ){
            Text(
                text = subject.name,
                style = MaterialTheme.typography.headlineMedium,
                fontWeight = FontWeight.Bold
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = "Преподаватель: ${subject.professor}",
                style = MaterialTheme.typography.titleMedium
            )
        }
    }
}

// Карточка с параметрами
Card(
    modifier = Modifier.fillMaxWidth()
){
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ){
        DetailRow(
            label = "Баллы",
            value = subject.credits.toString()
        )
        HorizontalDivider()
        DetailRow(
            label = "Текущая оценка",
            value = subject.currentGrade
        )
    }
}

// Карточка с описанием
Card(
    modifier = Modifier.fillMaxWidth()
){
    Column(
        modifier = Modifier.padding(16.dp)
    ){
        Text(
            text = "Описание курса",
            style = MaterialTheme.typography.titleMedium,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(8.dp))
        Text(
            text = subject.description,
            style = MaterialTheme.typography.bodyMedium,
            lineHeight = MaterialTheme.typography.bodyMedium.lineHeight.times(1.4f)
        )
    }
}

```

```

    )
  }
}
}
}
}

//Строка с меткой и значением
@Composable
fun DetailRow(
  label: String,
  value: String,
  modifier: Modifier = Modifier
) {
  Row(
    modifier = modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
  ) {
    Text(
      text = label,
      style = MaterialTheme.typography.bodyLarge,
      color = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Text(
      text = value,
      style = MaterialTheme.typography.bodyLarge,
      fontWeight = FontWeight.Medium
    )
  }
}
}

```

Объяснение:

- Параметры экрана:
 - `subjectId` — строка с ID дисциплины (передаётся через навигацию)
 - `onNavigateBack` — функция для возврата на предыдущий экран
- `find { it.id == subjectId }` — поиск дисциплины в списке по ID. Возвращает дисциплину или `null`
- Обработка случая, когда дисциплина не найдена (например, передан неверный ID)
- `navigationIcon` в `TopAppBar` — кнопка "Назад" (стрелка влево)
- Первая карточка — название дисциплины и преподаватель на цветном фоне
- Вторая карточка — параметры (кредиты, оценка) в виде строк "ключ-значение"
- Третья карточка — подробное описание курса

DetailRow:

- Вспомогательная функция для отображения строки вида "Метка: Значение" с выравниванием по краям

Скриншоты для отчёта



Сделайте скриншот:

- Файл **DetailsScreen.kt** с кодом
- Сохраните как: **step5_details_screen_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 5: Create DetailsScreen with subject information"

git push

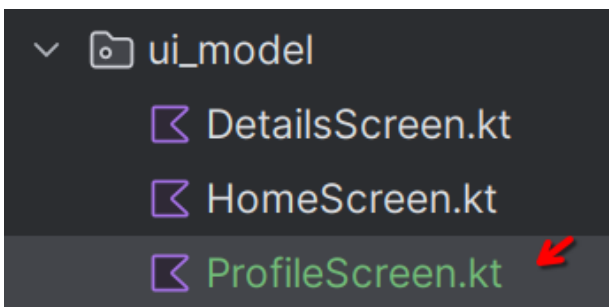
Шаг 6. Создание экранов Profile и Settings

Создадим два простых экрана для демонстрации навигации без параметров.

6.1. Создание файла ProfileScreen

1. Правой кнопкой на пакет **ui_model** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **ProfileScreen**
4. Нажмите **Enter**

Итог:



Добавьте следующий код (**обратите внимание на ваши пакеты**):

```
package com.ваш_пакет.studentplanner.ui_model

import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Star
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```



```
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

//Экран профиля студента
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProfileScreen(
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Профиль студента") },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(
                            imageVector = Icons.Default.ArrowBack,
                            contentDescription = "Назад"
                        )
                    }
                }
            )
        }
    ) {
        paddingValues ->
        Column(
            modifier = modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // Карточка с аватаром и именем
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer
                )
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(24.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Icon(
                        imageVector = Icons.Default.Person,
                        contentDescription = null,
                        modifier = Modifier.size(80.dp),
                        tint = MaterialTheme.colorScheme.primary
                    )
                    Spacer(modifier = Modifier.height(16.dp))
                    Text(
                        text = "Иванов Иван Иванович",
                        style = MaterialTheme.typography.headlineSmall,
                        fontWeight = FontWeight.Bold
                    )
                }
            }
        }
    }
}
```

```

    )
    Text(
      text = "Группа: ИСП-234",
      style = MaterialTheme.typography.titleMedium,
      color = MaterialTheme.colorScheme.onPrimaryContainer
    )
  }
}

// Информация о студенте
Card(modifier = Modifier.fillMaxWidth()) {
  Column(
    modifier = Modifier.padding(16.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp)
  ) {
    ProfileInfoRow(
      icon = Icons.Default.Star,
      label = "Университетский колледж",
      value = "Информационные системы и программирование"
    )
    HorizontalDivider()
    ProfileInfoRow(
      icon = Icons.Default.Email,
      label = "Email",
      value = "ivanov@vgi.volsu.ru"
    )
    HorizontalDivider()
    ProfileInfoRow(
      icon = Icons.Default.Person,
      label = "Курс",
      value = "3 курс"
    )
  }
}

// Статистика
Card(modifier = Modifier.fillMaxWidth()) {
  Column(
    modifier = Modifier.padding(16.dp)
  ) {
    Text(
      text = "Академическая успеваемость",
      style = MaterialTheme.typography.titleMedium,
      fontWeight = FontWeight.Bold
    )
    Spacer(modifier = Modifier.height(16.dp))
    Row(
      modifier = Modifier.fillMaxWidth(),
      horizontalArrangement = Arrangement.SpaceEvenly
    ) {
      StatisticItem(
        label = "Средний балл",
        value = "4.7"
      )
      StatisticItem(
        label = "Бонусов",
        value = "180"
      )
    }
  }
}

```

```

        StatisticItem(
            label = "Дисциплин",
            value = "20"
        )
    }
}
}
}
}

//Строка с иконкой, меткой и значением
@Composable
fun ProfileInfoRow(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    label: String,
    value: String,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.primary
        )
        Spacer(modifier = Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                text = label,
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
            Text(
                text = value,
                style = MaterialTheme.typography.bodyLarge,
                fontWeight = FontWeight.Medium
            )
        }
    }
}

//Элемент статистики
@Composable
fun StatisticItem(
    label: String,
    value: String,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(

```

```

        text = value,
        style = MaterialTheme.typography.headlineMedium,
        fontWeight = FontWeight.Bold,
        color = MaterialTheme.colorScheme.primary
    )
    Text(
        text = label,
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}

```

Объяснение:

- Экран профиля показывает информацию о студенте: имя, группу, факультет, email
- Использует те же принципы, что и предыдущие экраны: Scaffold, TopAppBar, Card
- ProfileInfoRow — переиспользуемый компонент для строки информации с иконкой
- StatisticItem — компонент для отображения статистики (средний балл, кредиты)

6.2. Создание файла SettingsScreen

1. Правой кнопкой на пакет **ui_model** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **SettingsScreen**
4. Нажмите **Enter**

Добавьте следующий код (**обратите внимание на ваши пакеты**):

```

package com.ваш_пакет.studentplanner.ui_model

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

//Экран настроек приложения
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SettingsScreen(

```

```

onNavigateBack: () -> Unit,
modifier: Modifier = Modifier
) {
    var notificationsEnabled by remember { mutableStateOf(true) }
    var darkModeEnabled by remember { mutableStateOf(false) }
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Настройки") },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(
                            imageVector = Icons.Default.ArrowBack,
                            contentDescription = "Назад"
                        )
                    }
                }
            )
        }
    )
}
) { paddingValues ->
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(paddingValues)
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // Секция "Уведомления"
        Card(modifier = Modifier.fillMaxWidth()) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = "Уведомления",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(12.dp))
                SettingsSwitchItem(
                    icon = Icons.Default.Notifications,
                    title = "Push-уведомления",
                    description = "Получать уведомления о дедлайнах",
                    checked = notificationsEnabled,
                    onCheckedChange = { notificationsEnabled = it }
                )
            }
        }
        // Секция "Внешний вид"
        Card(modifier = Modifier.fillMaxWidth()) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = "Внешний вид",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(12.dp))
                SettingsSwitchItem(
                    icon = Icons.Default.Lock,

```

```

        title = "Тёмная тема",
        description = "Использовать тёмное оформление",
        checked = darkModeEnabled,
        onCheckedChange = { darkModeEnabled = it }
    )
}
}
// Секция "Дополнительно"
Card(modifier = Modifier.fillMaxWidth()) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(
            text = "Дополнительно",
            style = MaterialTheme.typography.titleMedium,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(12.dp))
        SettingsMenuItem(
            icon = Icons.Default.Info,
            title = "О приложении",
            onClick = { /* TODO */ }
        )
        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))
        SettingsMenuItem(
            icon = Icons.Default.Phone,
            title = "Справка",
            onClick = { /* TODO */ }
        )
        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))
        SettingsMenuItem(
            icon = Icons.Default.ExitToApp,
            title = "Выход",
            onClick = { /* TODO */ }
        )
    }
}
}
}
}
}
// Элемент настройки с переключателем
@Composable
fun SettingsSwitchItem(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    title: String,
    description: String,
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = icon,

```

```

        contentDescription = null,
        tint = MaterialTheme.colorScheme.primary
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            text = title,
            style = MaterialTheme.typography.bodyLarge,
            fontWeight = FontWeight.Medium
        )
        Text(
            text = description,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Switch(
        checked = checked,
        onCheckedChange = onCheckedChange
    )
}
}

// Элемент настройки с кликом
@Composable
fun SettingsMenuItem(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    title: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.primary
        )
        Spacer(modifier = Modifier.width(12.dp))
        Text(
            text = title,
            style = MaterialTheme.typography.bodyLarge,
            modifier = Modifier.weight(1f)
        )
        Icon(
            imageVector = Icons.Default.Build,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}
}

```

Объяснение:

- Используем `remember { mutableStateOf() }` для хранения состояния переключателей
- `SettingsSwitchItem` — элемент настройки с переключателем (Switch)
- `SettingsMenuItem` — кликабельный элемент настройки со стрелкой вправо
- Экран организован по секциям: "Уведомления", "Внешний вид", "Дополнительно"

Скриншоты для отчёта



Сделайте скриншоты:

- Файл **ProfileScreen.kt**
- Файл **SettingsScreen.kt**
- Сохраните как: **step6_profile_settings_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 6: Create ProfileScreen and SettingsScreen"

git push

Шаг 7. Настройка NavHost и навигации

Теперь соберём все экраны вместе с помощью **Navigation Compose**.

7.1. Создание файла Navigation

1. Правой кнопкой на пакет **navigation** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **NavGraph**
4. Нажмите **Enter**

7.2. Создание NavHost

Что мы делаем: Создаём центральную точку навигации приложения, которая определяет все маршруты и связывает их с экранами.

Добавьте в начале файла следующие импорты:

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.navArgument
import com.ваш_пакет.studentplanner.ui_model.*
```


Напишем код

Определяем `@Composable`-функцию `StudentPlannerNavHost()`:

```
@Composable
fun StudentPlannerNavHost(
    navController: NavHostController,
    modifier: Modifier = Modifier
) { }
```

- **navController** — контроллер навигации, управляет переходами
- **modifier** — стандартный модификатор Compose

Внутри тела функции определяем функцию `NavHost()` - контейнер для всех экранов::

```
fun StudentPlannerNavHost(...) {
    NavHost(
        navController = navController,
        startDestination = Screen.Home.route,
        modifier = modifier
    ) { }
}
```

- **navController** — привязываем контроллер
- **startDestination** — начальный экран (Home)
- **modifier** — передаём модификатор дальше

Внутри тела функции `NavHost()` регистрируем наши экраны:

```
NavHost(...) {
    1 // Главный экран (список дисциплин)
    composable(route = Screen.Home.route) {...}
    2 // Экран деталей дисциплины (с параметром subjectId)
    composable(...) {...}
    3 // Экран профиля
    composable(route = Screen.Profile.route) {...}
    4 // Экран настроек
    composable(route = Screen.Settings.route) {...}
}
```

1. Главный экран (список дисциплин)

```
// Главный экран (список дисциплин)
composable(route = Screen.Home.route) {
    HomeScreen(
        onSubjectClick = { subjectId ->
            // Переход на экран деталей с передачей ID
            navController.navigate(Screen.Details.createRoute(subjectId))
        },
        onProfileClick = {
            // Переход на экран профиля
            navController.navigate(Screen.Profile.route)
        },
        onSettingsClick = {
            // Переход на экран настроек
            navController.navigate(Screen.Settings.route)
        }
    )
}
```

- **composable(route = Screen.Home.route)** — регистрируем маршрут "home"
- **onSubjectClick** — при клике вызываем `navController.navigate()` с ID
- **Screen.Details.createRoute(subjectId)** — создаёт строку вида "details/123"

2. Экран деталей дисциплины (с параметром subjectId)

```
// Экран деталей дисциплины (с параметром subjectId)
composable(
    route = Screen.Details.route,
    arguments = listOf(
        navArgument("subjectId") {
            type = NavType.StringType
        }
    )
) { backStackEntry ->
    // Извлекаем параметр subjectId из аргументов навигации
    val subjectId = backStackEntry.arguments?.getString("subjectId") ?: ""
    DetailsScreen(
        subjectId = subjectId,
        onNavigateBack = {
            // Возврат на предыдущий экран
            navController.popBackStack()
        }
    )
}
```

- **arguments = listOf(...)** — объявляем, что маршрут принимает параметр

- `navArgument("subjectId")` — имя параметра и его тип (String)
- `backStackEntry.arguments?.getString("subjectId")` — извлекаем значение параметра

- `navController.popBackStack()` — возврат назад (убирает текущий экран из стека)

3. Экран профиля

```
// Экран профиля
composable(route = Screen.Profile.route) {
    ProfileScreen(
        onNavigateBack = {
            navController.popBackStack()
        }
    )
}
```

4. Экран настроек

```
// Экран настроек
composable(route = Screen.Settings.route) {
    SettingsScreen(
        onNavigateBack = {
            navController.popBackStack()
        }
    )
}
```

Как работает навигация с параметрами:

1. Пользователь кликает на карточку дисциплины с ID = "2"
2. HomeScreen вызывает: `onSubjectClick("2")`
3. NavHost вызывает: `navController.navigate("details/2")`
4. NavController:
 - Парсит маршрут `"details/2"`
 - Находит соответствие с `"details/{subjectId}"`
 - Извлекает `subjectId = "2"`
 - Добавляет Details в back stack
5. NavHost отображает DetailsScreen с `subjectId = "2"`

7.3. Обновление MainActivity

Откройте файл MainActivity.kt.

Удалите неиспользуемый код, и замените содержимое **setContent()**:

```
setContent {  
    StudentPlannerTheme {  
        // Создаём NavController  
        val navController = rememberNavController()  
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
            // Передаём NavController в NavHost  
            StudentPlannerNavHost(  
                navController = navController,  
                modifier = Modifier.padding(innerPadding)  
            )  
        }  
    }  
}
```

- **rememberNavController()** — создаёт **NavController**. **remember** сохраняет его при рекомпозиции
- Передаём **navController** в наш **NavHost**

Важно: **rememberNavController()** должен вызываться на уровне Composable-функции, а не внутри onCreate напрямую.

Необходимые импорты:

```
import android.os.Bundle  
import androidx.activity.ComponentActivity  
import androidx.activity.compose.setContent  
import androidx.activity.enableEdgeToEdge  
import androidx.compose.foundation.layout.fillMaxSize  
import androidx.compose.foundation.layout.padding  
import androidx.compose.material3.Scaffold  
import androidx.compose.ui.Modifier  
import androidx.navigation.compose.rememberNavController  
import com.ваш_пакет.studentplanner.navigation.StudentPlannerNavHost  
import com.ваш_пакет.studentplanner.ui.theme.StudentPlannerTheme
```

7.4. Запуск и тестирование приложения

1. Нажмите **Run** (зелёная кнопка play) или **Shift + F10**
2. Выберите эмулятор или подключённое устройство
3. Дождитесь запуска приложения

Что должно работать:

Главный экран:

- Список из дисциплин
- Кнопки "Профиль" и "Настройки" в TopAppBar

Клик по дисциплине:

- Переход на экран деталей
- Отображается информация о выбранной дисциплине
- Кнопка "Назад" работает

Кнопка "Профиль":

- Переход на экран профиля
- Отображается информация о студенте
- Кнопка "Назад" возвращает на главный экран

Кнопка "Настройки":

- Переход на экран настроек
- Переключатели работают
- Кнопка "Назад" возвращает на главный экран

Системная кнопка "Назад":

- Корректно возвращает на предыдущие экраны
- При нажатии на главном экране — выход из приложения

Проверка Back Stack:

Попробуйте последовательность:

1. Home → Details → Back
2. Home → Profile → Back
3. Home → Details (дисциплина 1) → Back → Details (дисциплина 2) → Back

Скриншоты для отчёта



Сделайте скриншоты всех экранов:

1. Главный экран со списком дисциплин
2. Экран деталей (любая дисциплина)
3. Экран профиля
4. Экран настроек
5. Сохраните как: **step7_navigation_working_FIO_1.png** (добавьте в конце цифры или описание разных экранов)

Выполните в терминале:

git add .

git commit -m "Step 7: Setup NavHost and complete navigation"

git push

Самостоятельная работа

Задание 1: Создание README.md

Создайте файл README.md в корне проекта, который должен содержать:

1. Название проекта
2. Краткое описание приложения (2-3 предложения)
3. Список реализованных экранов
4. Используемые технологии (Kotlin, Jetpack Compose, Navigation Compose)
5. Схему навигации между экранами (текстовую)
6. Скриншоты основных экранов

Ответьте на следующие вопросы:

Контрольные вопросы

1. **Что такое NavController и для чего он используется?**
 - Объясните своими словами роль NavController в навигации
 - Почему важно создавать его через rememberNavController()?
2. **Как передать параметр в маршрут навигации?**
 - Опишите процесс: от определения маршрута до извлечения параметра
 - В чём разница между обязательными и опциональными параметрами?
3. **Зачем использовать sealed class для маршрутов?**
 - Какие преимущества даёт sealed class по сравнению с обычными строками?
 - Приведите пример ошибки, которую sealed class помогает избежать
4. **Что такое Back Stack и как им управлять?**
 - Нарисуйте схему back stack для последовательности: Home → Profile → Settings
 - Что произойдёт при вызове popBackStack() на экране Settings?
5. **Как работает startDestination в NavHost?**
 - Какой экран будет показан первым при запуске приложения?
 - Можно ли изменить startDestination динамически?

6. Что произойдёт, если навигировать на несуществующий маршрут?

- Как NavController обрабатывает неизвестные маршруты?
- Как можно обработать такую ситуацию?

7. Зачем нужен параметр launchSingleTop в навигации?

- Приведите пример, когда без него может возникнуть проблема
- Как он влияет на back stack?

После выполните в терминале:

git add README.md

git commit -m "Add project README with navigation description"

git push

Задание 2: Заполнение информации

1. Допишите список всеми дисциплинами, которые вы изучаете в текущем семестре, с описанием, ФИО преподавателя, оценкой.
2. В Профиле студента добавьте информацию о себе.

Задание 3: Расширение функциональности

Добавьте экран "Расписание"

- Создайте новый экран с расписанием занятий на неделю
- Добавьте кнопку перехода на главном экране
- Реализуйте навигацию к деталям занятия при клике

Требования к самостоятельной работе:

- Сделайте осмысленный git commits
- Обновите README с описанием новой функциональности